
A UNIFIED FRAMEWORK FOR AUTOMATED DISRUPTION LABELING OF DIII-D SHOTS USING A 1D CONVOLUTIONAL NEURAL NETWORK AND TEMPORAL FILTERING

Jason Jain, Dr. Jesse Rodriguez
Oregon State University
Coupled Systems Laboratory

July 24, 2026

ABSTRACT

Plasma in a Tokamak fusion reactor frequently experience disruption events. The existence of a computationally inexpensive model for identifying disruptive shots and accurately identifying time of disruption can assist the scientific community by auto-labeling shots for further analysis. We took a labeled disruption dataset from the DIII-D Tokamak reactor, utilizing shot numbers 125500-186000 (nearly 40,000 total shots) to develop a new disruption labeling model that can generate automated labeling up to the current day. We train a CNN (Convolutional Neural Network) to perform boolean disruption/not-disruption prediction and achieve a 99% recall and 91% precision on our test dataset. We additionally develop a convolutional heuristic that accurately identifies time of disruption within a standard deviation of 1.1 ms.

1 Introduction

A Tokamak is a torus-shaped magnetic confinement fusion reactor and is a leading candidate to produce controlled thermonuclear fusion power. A prominent challenge facing Tokamak reactors is the issue of disruption events caused by turbulent instability within the plasma. Such disruption events cause the internal reactor current ($I(t)$) to drop to zero and can potentially cause damage to the interior of the reactor. The differential equations describing the evolution of such turbulent plasma are not solvable with conventional numerical methods; therefore, deep learning models trained on labeled plasma data are a promising approach for predicting such disruptions before they happen.

A fraction of the data produced by long-running Tokamak fusion facilities such as the DIII-D reactor in Argonne National Laboratory have already been labeled for training of such models. We used nearly 40,000 DIII-D shots to train a model for labeling historical shots as disruptive or not disruptive. We also develop a convolutional heuristic for identifying a time of disruption that closely matches the labeled DIII-D disruption times.

2 Neural Net Classifier

We were provided 39438 labeled shot files from the DIII-D team, of which 13429 were disruptive shots and labeled with time of disruption. Each shot sampled reactor current at 40 kHz, giving us a discrete $1 \times m$ current vector $\mathbf{I}$ where the m th component is current at the m th timestep.

$$\mathbf{I} = [I_1 \quad \dots \quad I_m] \quad (1)$$

We used an 80/10/10 train/test/dev split, which left us with 31542 trainable shots. To prepare the data for a neural net classifier, we tested three different normalization methods, including per-shot min-max unit scale normalization, per-shot Mean-Variance (meanvar) normalization, and dataset-wide meanvar normalization. We found most success with the dataset-wide meanvar approach. Letting the current mean and standard deviation across all shots be μ and σ , we computed a normalized shot tensor $\mathbf{I}_N$.

$$\mathbf{I}_N = \begin{cases} \frac{1}{\sigma}(\mathbf{I} - \mu) & \sigma > 0 \\ 0 & \sigma \leq 0 \end{cases} \quad (2)$$

The normalized data was then fed into a Convolutional Neural Network (see Fig 1) consisting of four convolutional layers, each with a batch normalization layer, a

Rectified Linear Unit (ReLU) activation function, and a final max pooling layer. The convolution outputs were then fed into two fully connected layers with batch normalization, ReLU, and a dropout layer to prevent overfitting. The dropout layer exposed a dropout rate dropout hyperparameter.

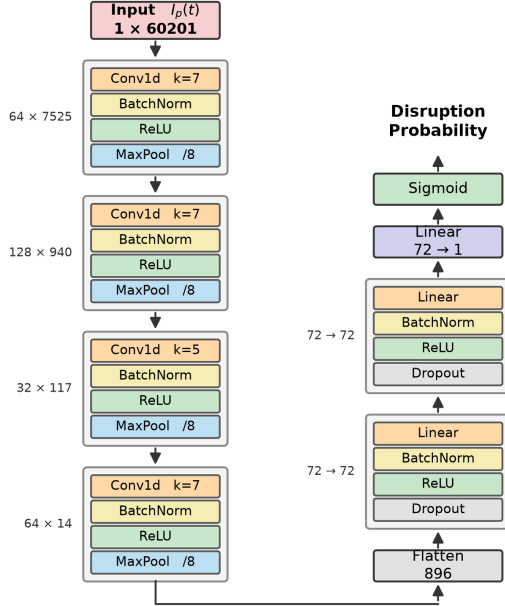


Figure 1: Model architecture for our disruption labeling Convolutional Neural Network. We have four convolutional layers and two fully connected layers, and utilize max pooling, batch normalization, and dropout, along with a ReLU activation function.

The CNN was then trained using the Adam (Adaptive Moment Estimation) optimizer with weight decay and a learning rate tuned per-epoch with PyTorch’s `lr_scheduler.ReduceLROnPlateau` in "min" mode, with configurable learning rate factor and patience. We utilized PyTorch’s `torch.nn.BCEWithLogitsLoss` with configurable positive weight and gradient clipping, and stopped training when enough consecutive epochs showed no improvement (early stopping patience).

We tuned our hyperparameters with Bayesian Optimization, using the ExpectedImprovement acquisition function with $\xi = 0.05$. We additionally inserted random samples every 5 trials to escape local minima. Our scoring strategy was a weighted F_β , where parameter β is the weighted harmonic mean of precision and recall Eq (3). We chose $\beta = 1.5$ to prioritize recall (R) over precision (P) and minimize false negatives, rejecting trials below a precision floor of 90%.

$$F_\beta = \frac{PR(1 + \beta^2)}{P\beta^2 + R} \quad (3)$$

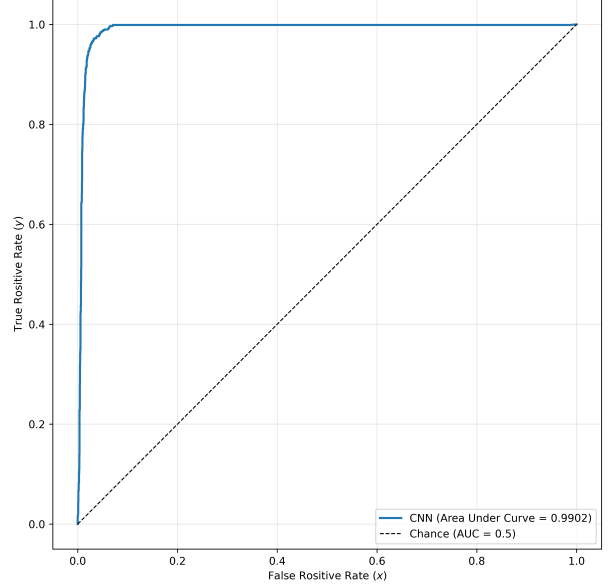


Figure 2: The ROC curve for our disruption classifier. The integral (AUC) of the ROC characterizes a classifier’s performance, with a score of 1 indicating 100% true positives and 0% false negatives. Our model has an AUC of 0.9902, close to the hypothetical maximum.

We trained for 100+ trials on the Polaris supercomputer in Argonne National Lab, achieving a final precision of 90.52% and recall of 98.84%, along with an F1 score of 0.9450 and F2 score of 0.9706. The tuned hyperparameters are presented in Table 1.

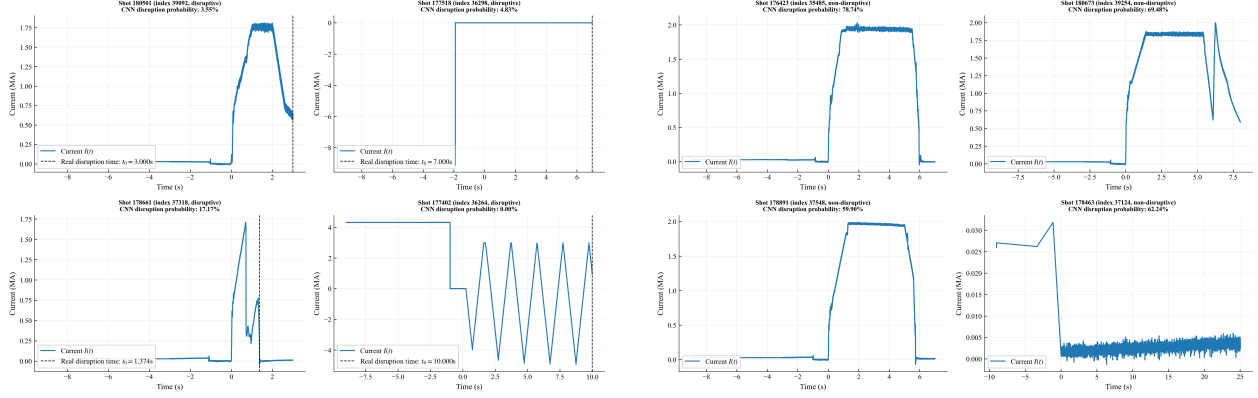
Table 1: Tuned Hyperparameters

Hyperparameter	Value
Learning Rate	0.0005
Number of Epochs	200
Dropout Rate	0.35
Weight Decay	0.0001
Training Batch Size	128
Gradient Clip	1.0
Learning Rate Scheduler Factor	0.5
Learning Rate Patience	5
Early Stopping Patience	10
Positive Weight	3.0

The false negatives produced by the model all lack the characteristic drop to zero, and some look like faulty data. See Fig 3.

3 Convolutional Heuristic for Time of Disruption

For any given shot, let the current be $I(t)$, $t \in [0, T]$. We start by noting that shots with $\max I(t) \approx 0$ are negative polarity shots and must be flipped before proceeding Eq (4).



(a) Four selected disruptive shots incorrectly identified as non-disruptive by the CNN. Each shot lacks the characteristic instant drop to zero that typically identifies a disruption. Shots 180501 and 177403 read like faulty data, as if the device stopped recording mid-experiment.

(b) Four selected non-disruptive shots incorrectly identified as disruptive by the CNN. Shots 178891 and 176423 have a drop that somewhat resembles a disruption, confusing the model. Notice that the CNN disruption probability is lower for the more confusing shots.

Figure 3: Analysis of CNN classification errors for disruptive (Fig 3a) and non-disruptive (Fig 3b) shots.

$$I_{\text{raw}}(t) = \begin{cases} -I(t) & \max I \approx 0 \\ I(t) & \text{otherwise} \end{cases} \quad (4)$$

We then calculate a smoothed current $I_{\text{smooth}}(t)$ by applying a moving average convolution with a window size $K = T/S$, where $S = 300$ is an experimentally determined smoothing parameter. The convolution of $I(t)$ with a function $g(t)$ is given by $g * f(t) = \int_0^t g(\tau) f(t - \tau) d\tau$; for a moving average, g is a constant, simplifying our expression to Eq (5).

$$I_{\text{smooth}}(t) = \frac{1}{K} \int_{t-K}^t I(\tau) d\tau \quad (5)$$

Note that we compute the moving average on our dataset as a discrete convolution, applying `np.convolve` on the current tensor Eq (1). The i th component of the smooth current I_{smooth} is $I_{\text{smooth}_i} = \frac{1}{K} \sum_{k=1}^K I_{i-k}$.

We then define the time of disruption heuristic $f(t)$ as the difference between the raw $I(t)$ and smooth current Eq (5).

$$f(t) = I_{\text{raw}}(t) - I_{\text{smooth}}(t) \quad (6)$$

To find the start $t = t_0$ of a disruption, we simply search for the global maximum of $f(t)$. The first local minimum of $f(t)$, $t > t_0$ identifies the end of disruption t_f .

$$t_0 = \max(f(t)) \quad (7)$$

We apply this heuristic on our test holdout set, consisting of 3943 total shots, of which 889 are disruptive.

Comparing t_0 against the labeled time of disruption t_D over our test set, we find that most of our predictions are early by $\Delta t = t_0 - t_D \approx -4$ ms (Fig 5b). Therefore, we conclude that the DIII-D team is evidently applying

Shot 139555 (index 9543, disruptive)

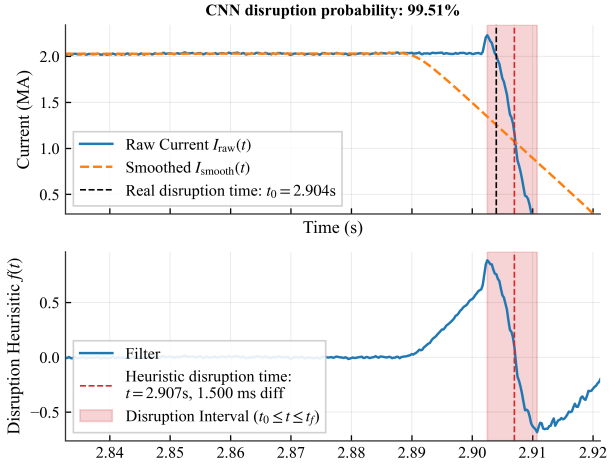


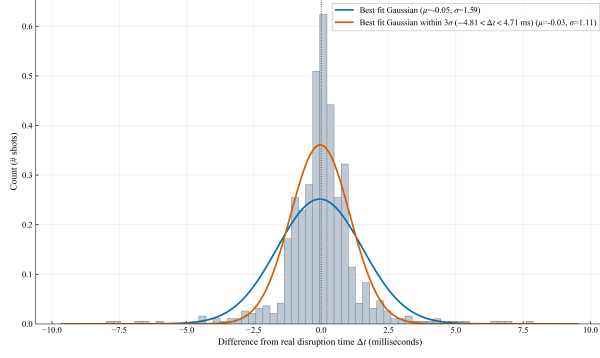
Figure 4: We see the raw Eq (4) and smooth Eq (5) currents above, and the convolutional heuristic $f(t)$ Eq (6) below, both graphed versus time. The absolute maximum of $f(t)$ yields the start of disruption $t = t_0$ Eq (7), with the following minimum giving the end of disruption t_f . The root t_{root} Eq (8) gives an estimate closer to DIII-D's disruption labels (Fig 5a).

labels mid-disruption, somewhere in the range $t_0 \leq t \leq t_f$. If we use the first root t_{root} Eq (8) as the predicted time of disruption, we get a more normalized distribution of $\Delta t = t_{\text{root}} - t_D$ Fig 5a.

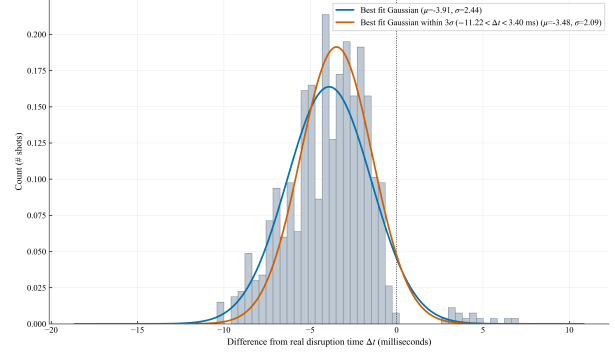
$$f(t_{\text{root}}) = 0, \quad t_{\text{root}} \in [t_0, t_f] \quad (8)$$

4 Conclusion

Idk conclude somethin or whatever



(a) Histogram of the difference Δt between heuristic root t_{Toot} Eq (8) and labeled time of disruption in milliseconds. The Gaussian is centered with $\mu \approx 0$ ms, $\sigma = 1.1$ ms



(b) Histogram of the difference Δt between predicted start of disruption Eq (7) and labeled time of disruption in milliseconds. Most of our predictions are early, with a mean $\Delta t \approx -4$ ms

Figure 5: Histogram of Fig 5a and non-disruptive Fig 5b shots.

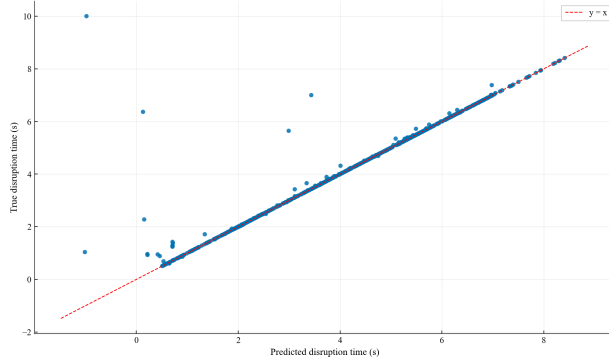


Figure 6: Predicted disruption time Eq (7) vs real disruption time for 889 test holdout shots. We see a few outliers, with the majority of predictions on the accurate $y = x$ line.

Acknowledgments

Our thanks to Argonne National Laboratory for providing us with the data to train our model.

Data and code availability

Code for the model, along with the best model weights and hyperparameters, are hosted at <https://github.com/starlightjason2/DLTL.git>.

References

- [1] Anthropic. Claude code (opus 4.8). <https://www.anthropic.com/claude-code>, 2026. AI assistant used for code development and LaTeX formatting.